



EFFICIENT AI ON YOUR OWN HARDWARE

Small Language Models in Practice

A hands-on, code-first guide to fine-tuning, RAG, agents, quantization, and local deployment of compact language models.

WRITTEN BY

Haji Gul

Small Language Models in Practice

Efficient AI on Your Own Hardware

Copyright © 2026 Haji Gul.

All code listings in this book are provided for educational use. Model and dataset identifiers refer to public hubs and may change over time; verify current names before running. The author makes no warranty as to the fitness of any code for a particular purpose.

Typeset with $\text{\LaTeX}$. First Edition, 2026.

Contents

Preface	ii
1 Why Small Language Models	1
1.1 The case for going small	1
1.2 How small is small enough?	1
1.3 The stack we will use	2
1.4 Hello, small model	2
1.5 Recap and exercise	3
2 Running Models Locally	4
2.1 Two paths: library vs. runner	4
2.2 The transformers path, properly	4
2.3 Streaming output	5
2.4 The Ollama path	6
2.5 Recap and exercise	6
3 Fine-Tuning with LoRA	7
3.1 Why and when to fine-tune	7
3.2 Step 1: prepare the data	7
3.3 Step 2: load model and tokenizer	8
3.4 Step 3: attach LoRA adapters	8
3.5 Step 4: train	8
3.6 Step 5: save and reuse the adapter	9
3.7 Recap and exercise	10
4 Retrieval-Augmented Generation	11
4.1 The idea in one paragraph	11
4.2 Step 1: chunk your documents	11
4.3 Step 2: embed the chunks	12
4.4 Step 3: store them in a vector database	12
4.5 Step 4: retrieve relevant context	12
4.6 Step 5: generate the grounded answer	13
4.7 Recap and exercise	13
5 Agents and Tool Use	14
5.1 What an agent really is	14
5.2 Step 1: define some tools	14
5.3 Step 2: the tool-calling loop	15
5.4 Step 3: watch it work	16
5.5 Using a framework	16
5.6 Recap and exercise	16

6	Quantization for Small Hardware	17
6.1	The memory problem	17
6.2	4-bit loading with bitsandbytes	17
6.3	GGUF for llama.cpp and Ollama	18
6.4	Recap and exercise	19
7	Serving and Deployment	20
7.1	From script to service	20
7.2	A FastAPI generation service	20
7.3	Streaming responses	21
7.4	The no-code path: Ollama serve	22
7.5	Recap and exercise	22
8	Capstone: A Local Document Assistant	23
8.1	What we are building	23
8.2	Part 1: the quantized model	23
8.3	Part 2: the retrieval index	24
8.4	Part 3: the tool	24
8.5	Part 4: the orchestrator	24
8.6	Part 5: serve it	25
8.7	Where to go next	26

Preface

The last few years made one thing obvious: you do not need a billion-dollar cluster to ship useful language-model features. A 0.5B–7B parameter model, fine-tuned on the right data and quantized to fit a laptop GPU, will out-serve a giant API model on a narrow task — cheaper, faster, private, and entirely under your control. That is what this book is about.

This is a coding book. Every chapter is built around runnable Python. The theory is here too, but only enough to understand *why* a line of code exists; the moment a concept is clear, we write the code that uses it. If you prefer learning by typing, you are in the right place.

Who this is for

You can write Python and you have used `pip`. You do not need a machine learning PhD. If you have heard of transformers, tokens, and GPUs but have never fine-tuned, quantized, or deployed a model yourself, this book takes you from “I have heard of it” to “I shipped it.”

What you will build

By the final chapter you will have, with your own hands:

- run a small model locally with Ollama and the Hugging Face stack;
- fine-tuned a compact model on a domain dataset using LoRA;
- built a retrieval-augmented generation (RAG) pipeline over your own documents;
- wired a model into a tool-using agent;
- quantized a model to 4-bit so it fits on modest hardware;
- served a model behind a FastAPI endpoint; and
- combined all of the above into one end-to-end local assistant.

How to read it

Read in order the first time — later chapters reuse code and ideas from earlier ones. Each chapter opens with a short *In this chapter* box and ends with a recap and a small exercise. Code that you run in a shell appears in dark boxes; Python appears in light boxes labelled Python.

Conventions and current trends

The ecosystem moves fast. This edition targets the stack as it stabilized for local and domain-specific work: Hugging Face **transformers**, **peft** for LoRA, **bitsandbytes** and GGUF for quantization, **llama.cpp** and Ollama for local serving, LanceDB for vectors, and lightweight agent frameworks. Where a model or dataset name might drift, the text says so. The accompanying Jupyter *lecture notes* mirror every listing so you can run them directly.

— Haji Gul, 2026

Why Small Language Models

In this chapter

Why “small” is often the right choice; the size/quality trade-off in plain terms; the local-first stack we will use; and a 10-line program that proves your environment works.

1.1 The case for going small

A large frontier model is a generalist: it knows a little about everything and costs a lot to call. Most real products do not need a generalist. They need a model that is excellent at *one* domain — your support tickets, your legal clauses, your sensor logs — and that runs where your data already lives.

A **small language model (SLM)** — here, roughly 0.3B to 8B parameters — gives you four things a giant API model cannot:

- **Privacy.** The data never leaves your machine.
- **Cost.** After download, inference is essentially free.
- **Latency.** No network round-trip; tokens stream from local memory.
- **Control.** You pin the weights, the version, and the behavior.

The core trade-off

A smaller model knows less out of the box. We close that gap three ways, each a chapter in this book: **fine-tuning** (teach it your domain), **retrieval** (give it your documents at query time), and **agents** (let it call tools). Quantization and serving then make it cheap to run.

1.2 How small is small enough?

There is no single answer, but a useful rule of thumb for 2026-era models:

Size	Runs on	Good for
0.3–1B	Any laptop CPU	Classification, extraction, simple chat
1–3B	Laptop GPU / strong CPU	Domain Q&A, RAG, code snippets
3–8B	8–16 GB GPU (quantized)	Agents, reasoning, capable assistants

Start as small as the task allows. You can always scale up; you rarely need to.

1.3 The stack we will use

Everything in this book is open source and runs locally. The setup cell you will reuse across chapters:

```
## Core
pip install transformers datasets accelerate
pip install torch # pick the build matching your CPU/CUDA

# Fine-tuning, RAG, quantization, serving (added per chapter)
pip install peft bitsandbytes
pip install lancedb sentence-transformers
pip install fastapi uvicorn
```

1.4 Hello, small model

Let us prove the toolchain works. This loads a tiny instruction-tuned model and generates a reply — on CPU, in well under a minute after the first download.

Python

```
from transformers import pipeline

# A small, capable instruct model. Swap for any HF model id you like.
gen = pipeline(
    "text-generation",
    model="Qwen/Qwen2.5-0.5B-Instruct",
    device_map="auto", # uses GPU if present, else CPU
)

messages = [
    {"role": "system", "content": "You are a concise assistant."},
    {"role": "user", "content": "Explain what a small language model is in one sentence."},
]

out = gen(messages, max_new_tokens=60, do_sample=False)
print(out[0]["generated_text"][-1]["content"])
```

Tip

If the download is slow or the model id has changed, any small instruct model works here, e.g. `HuggingFaceTB/SmolLM2-360M-Instruct` or `meta-llama/Llama-3.2-1B-Instruct`. The rest of the book is written so you can substitute models freely.

1.5 Recap and exercise

You learned *when* an SLM beats a giant model, how to pick a size, and you ran your first local generation.

Exercise. Change the user message to a question from your own domain and compare a 0.5B model with a 1.5B one. Note the difference in answer quality and in generation speed — that trade-off is the theme of this entire book.

Running Models Locally

In this chapter

Two ways to run an SLM on your own machine — the Hugging Face stack (maximum control) and Ollama (maximum convenience); tokenization and sampling knobs that actually matter; and streaming output token-by-token.

2.1 Two paths: library vs. runner

You will use both throughout the book:

- **Hugging Face transformers** — a Python library. Full control over tokenization, generation, fine-tuning, and quantization. This is what we fine-tune and quantize in later chapters.
- **Ollama** — a local model *runner* with a one-line install and a tiny API. Best when you just want a model serving on `localhost` without writing loader code.

2.2 The transformers path, properly

The `pipeline` helper from Chapter 1 is convenient but hides the two objects you will work with constantly: the **tokenizer** and the **model**. Loading them yourself is worth understanding.

Python

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "Qwen/Qwen2.5-0.5B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.float16,  # half precision halves memory
    device_map="auto",
)
```

The tokenizer turns text into integer *token* ids the model understands, and back again. Chat models expect a specific prompt format; never hand-format it — use the built-in chat template.

Python

```
messages = [
    {"role": "system", "content": "You are a helpful, concise assistant."},
    {"role": "user", "content": "Give me three uses for a local SLM."},
]

# Apply the model's own chat template, then tokenize.
inputs = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,    # signals "your turn, model"
    return_tensors="pt",
).to(model.device)
```

Now generate. The sampling arguments are the dials you will tune most often.

Python

```
output_ids = model.generate(
    inputs,
    max_new_tokens=200,
    do_sample=True,
    temperature=0.7,    # lower = more focused, higher = more creative
    top_p=0.9,          # nucleus sampling
    repetition_penalty=1.1,
)

# Decode only the newly generated tokens, not the prompt.
reply = tokenizer.decode(
    output_ids[0][inputs.shape[-1]:],
    skip_special_tokens=True,
)
print(reply)
```

The three sampling dials

temperature flattens or sharpens the probability distribution. **top_p** keeps only the smallest set of tokens whose probabilities sum to **p**. **repetition_penalty** discourages loops. For factual tasks set **do_sample=False** (greedy) and ignore the rest.

2.3 Streaming output

Waiting for a long answer is painful. Stream tokens as they are produced.

Python

```
from transformers import TextStreamer

streamer = TextStreamer(tokenizer, skip_prompt=True, skip_special_tokens=True)

_ = model.generate(
```

```

inputs,
max_new_tokens=200,
do_sample=True,
temperature=0.7,
streamer=streamer,    # prints tokens to stdout as they arrive
)

```

2.4 The Ollama path

Ollama installs as a desktop/CLI app. Once running, pulling and chatting with a model is two commands:

```

## After installing Ollama from ollama.com
ollama pull qwen2.5:0.5b
ollama run qwen2.5:0.5b "Give me three uses for a local SLM."

```

It also exposes an OpenAI-compatible HTTP API on port 11434, which we use in the deployment chapter. Calling it from Python needs no special SDK:

Python

```

import requests

resp = requests.post(
    "http://localhost:11434/api/chat",
    json={
        "model": "qwen2.5:0.5b",
        "messages": [{"role": "user", "content": "One sentence: what is RAG?"}],
        "stream": False,
    },
)
print(resp.json()["message"]["content"])

```

Tip

Use **Ollama** for quick local serving and demos; use **transformers** whenever you need to fine-tune, quantize, or inspect internals. The rest of the book mostly takes the transformers path because that is where the control is.

2.5 Recap and exercise

You can now load a model and tokenizer explicitly, format chat prompts correctly, control sampling, stream output, and run the same model through Ollama.

Exercise. Generate the same prompt at `temperature=0.2` and `temperature=1.2`. Describe how the answers change. Which would you pick for a customer-facing FAQ bot, and why?

Fine-Tuning with LoRA

In this chapter

When fine-tuning beats prompting; why LoRA makes it cheap; and a complete, runnable workflow that adapts a small model to your domain on a single GPU — data prep, training, and saving the adapter.

3.1 Why and when to fine-tune

Fine-tuning bakes new behavior into the weights. Reach for it when you want the model to adopt a *style*, a *format*, or *domain reflexes* that a prompt cannot reliably enforce — consistent JSON output, your support tone, a narrow classification. (When you instead need fresh *facts*, use retrieval; that is the next chapter.)

Full fine-tuning vs. LoRA

Full fine-tuning updates every weight — huge memory, easy to overfit on small data. **LoRA** (Low-Rank Adaptation) freezes the original weights and trains tiny “adapter” matrices instead. You update well under 1% of the parameters, it fits on a modest GPU, and the result is a small file you can swap in and out.

3.2 Step 1: prepare the data

We will adapt a small model to answer in a strict instruction/response format. Your dataset just needs an instruction and a target response per row. Here we load one from the Hub; swap in your own JSON/CSV the same way.

Python

```
from datasets import load_dataset

# Any instruction-style dataset works. Replace with your own:
# load_dataset("json", data_files="my_data.jsonl")
dataset = load_dataset("databricks/databricks-dolly-15k", split="train")
dataset = dataset.select(range(2000)) # keep it small for a quick run
```

```
def to_text(row):
    instr, ctx, resp = row["instruction"], row["context"], row["response"]
    prompt = instr if not ctx else f"{instr}\n\nContext: {ctx}"
    return {"text": f"### Instruction:\n{prompt}\n\n### Response:\n{resp}"}

dataset = dataset.map(to_text, remove_columns=dataset.column_names)
print(dataset[0]["text"][:300])
```

3.3 Step 2: load model and tokenizer

Python

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "Qwen/Qwen2.5-0.5B"  # base (non-instruct) model to adapt

tokenizer = AutoTokenizer.from_pretrained(model_id)
tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    model_id,
    torch_dtype=torch.float16,
    device_map="auto",
)
```

3.4 Step 3: attach LoRA adapters

The peft library wraps the model so only the adapters train.

Python

```
from peft import LoraConfig, get_peft_model

lora = LoraConfig(
    r=16,  # rank of the adapter (capacity)
    lora_alpha=32,  # scaling
    lora_dropout=0.05,
    target_modules=["q_proj", "v_proj"],  # attention projections
    task_type="CAUSAL_LM",
)

model = get_peft_model(model, lora)
model.print_trainable_parameters()  # shows the tiny % being trained
```

3.5 Step 4: train

We tokenize, then hand everything to the `Trainer`. The data collator builds language-modeling labels for us.

Python

```
def tokenize(batch):
    out = tokenizer(batch["text"], truncation=True, max_length=512)
    return out

tokenized = dataset.map(tokenize, batched=True, remove_columns=["text"])
```

Python

```
from transformers import (
    Trainer, TrainingArguments, DataCollatorForLanguageModeling,
)

args = TrainingArguments(
    output_dir="./qwen-lora",
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,    # effective batch size 16
    num_train_epochs=1,
    learning_rate=2e-4,
    fp16=True,
    logging_steps=25,
    save_strategy="epoch",
    report_to="none",
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=tokenized,
    data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False),
)

trainer.train()
```

3.6 Step 5: save and reuse the adapter

The adapter is only a few megabytes. Save it, then load it on top of the base model whenever you need your fine-tuned behavior.

Python

```
model.save_pretrained("./qwen-lora-adapter")
tokenizer.save_pretrained("./qwen-lora-adapter")
```

Python

```
from peft import PeftModel

base = AutoModelForCausalLM.from_pretrained(
    model_id, torch_dtype=torch.float16, device_map="auto"
)
tuned = PeftModel.from_pretrained(base, "./qwen-lora-adapter")
```

```
prompt = "### Instruction:\nList two benefits of local SLMs.\n\n### Response:\n"
ids = tokenizer(prompt, return_tensors="pt").to(tuned.device)
out = tuned.generate(**ids, max_new_tokens=80)
print(tokenizer.decode(out[0], skip_special_tokens=True))
```

Tip

To ship a single self-contained model, call `tuned.merge_and_unload()` to fold the adapter back into the weights, then `save_pretrained`. Keep the adapter separate while experimenting; merge only when you deploy.

3.7 Recap and exercise

You fine-tuned a base model on a domain dataset using LoRA, trained only a tiny fraction of parameters, and saved a reusable adapter.

Exercise. Replace the dataset with 50–100 of your own instruction/response pairs in a JSONL file and run one epoch. Compare the tuned model’s format-consistency against the base model on a held-out prompt.

Retrieval-Augmented Generation

In this chapter

Giving a small model fresh knowledge without retraining; embeddings and vector search explained simply; and a complete RAG pipeline over your own documents using LanceDB — chunk, embed, retrieve, answer.

4.1 The idea in one paragraph

A fine-tuned model still only knows what was in its training data. **RAG** fixes this at query time: store your documents as searchable vectors, retrieve the few chunks most relevant to the question, and paste them into the prompt as context. The model then answers *from your data* — no retraining, and you can update the knowledge base any time.

Embeddings

An **embedding** is a list of numbers representing a piece of text so that similar meanings land close together in space. To find relevant chunks we embed the question and return the stored chunks whose vectors are nearest. That nearest search is what a **vector database** does fast.

4.2 Step 1: chunk your documents

Long documents must be split into retrievable pieces. Overlap keeps sentences from being cut awkwardly across chunks.

Python

```
def chunk_text(text, size=500, overlap=50):
    chunks, start = [], 0
    while start < len(text):
        end = start + size
        chunks.append(text[start:end])
        start = end - overlap
    return chunks
```

```
docs = [
    "Small language models run locally and keep data private. ...",
    "LoRA trains tiny adapter matrices instead of full weights. ...",
    # In practice: read .txt/.md/.pdf files and concatenate their text.
]

chunks = [c for d in docs for c in chunk_text(d)]
print(f"{len(chunks)} chunks ready")
```

4.3 Step 2: embed the chunks

We use a small, fast sentence-embedding model that runs on CPU.

Python

```
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
vectors = embedder.encode(chunks, normalize_embeddings=True)
print(vectors.shape)  # (n_chunks, 384)
```

4.4 Step 3: store them in a vector database

LanceDB is file-based — no server to run. We store each chunk with its vector.

Python

```
import lancedb

db = lancedb.connect("./rag_db")
data = [
    {"text": chunk, "vector": vec.tolist()}
    for chunk, vec in zip(chunks, vectors)
]
table = db.create_table("docs", data=data, mode="overwrite")
```

4.5 Step 4: retrieve relevant context

Embed the question with the *same* model, then ask the table for the nearest chunks.

Python

```
def retrieve(question, k=3):
    q_vec = embedder.encode(question, normalize_embeddings=True)
    hits = table.search(q_vec.tolist()).limit(k).to_list()
    return [h["text"] for h in hits]

context = retrieve("How does LoRA save memory?")
for c in context:
    print("-", c[:80])
```

4.6 Step 5: generate the grounded answer

Stitch retrieved context into the prompt and let the model answer from it.

Python

```
from transformers import pipeline

gen = pipeline("text-generation",
               model="Qwen/Qwen2.5-0.5B-Instruct", device_map="auto")

def rag_answer(question):
    context = "\n\n".join(retrieve(question))
    messages = [
        {"role": "system",
         "content": "Answer using ONLY the context. If unsure, say so."},
        {"role": "user",
         "content": f"Context:\n{context}\n\nQuestion: {question}"},
    ]
    out = gen(messages, max_new_tokens=150, do_sample=False)
    return out[0]["generated_text"][-1]["content"]

print(rag_answer("How does LoRA save memory?"))
```

Tip

Two levers dominate RAG quality: **chunk size** (too big dilutes relevance, too small loses context) and **k** (how many chunks you retrieve). Tune these before reaching for a bigger model — they matter more.

4.7 Recap and exercise

You built a full RAG pipeline: chunk, embed, store, retrieve, and generate a grounded answer — all locally, with no retraining.

Exercise. Point the loader at a folder of your own `.txt` files, rebuild the table, and ask three domain questions. Add one question the documents do *not* answer and confirm the model admits it does not know.

Agents and Tool Use

In this chapter

Turning a passive model into something that *acts*; the tool-calling loop in its simplest honest form; and a working agent that decides when to call a calculator and a search function — built from scratch so you see every moving part.

5.1 What an agent really is

Strip away the hype and an **agent** is a loop: the model is told which tools exist, it responds with a request to call one, your code runs that tool and feeds the result back, and the loop repeats until the model produces a final answer. The model never runs anything itself — *you* do, on its instruction. That separation is the whole safety and control story.

The agent loop

1. Describe the tools to the model.
2. Model emits a tool call (name + arguments).
3. Your code executes the tool.
4. Feed the result back.
5. Repeat until the model answers instead of calling a tool.

5.2 Step 1: define some tools

A tool is just a Python function plus a description the model can read.

Python

```
import json

def calculator(expression: str) → str:
    """Evaluate a basic arithmetic expression."""
    allowed = set("0123456789+-*/(). ")
    if not set(expression) ≤ allowed:
        return "Error: invalid characters"
    return str(eval(expression)) # safe: input is character-restricted

def word_count(text: str) → str:
```

```

    """Count the words in a string."""
    return str(len(text.split()))

TOOLS = {"calculator": calculator, "word_count": word_count}

TOOL_SPEC = """
You can call tools by replying with ONLY a JSON object:
{"tool": "<name>", "args": {"<arg>": "<value>"}}
Available tools:
- calculator(expression): evaluate arithmetic, e.g. "12 * (3+4)"
- word_count(text): count words in text
When you have the final answer, reply with plain text (no JSON).
"""

```

5.3 Step 2: the tool-calling loop

We parse the model's reply: if it is JSON, run the tool and continue; otherwise it is the final answer.

Python

```

from transformers import pipeline

gen = pipeline("text-generation",
               model="Qwen/Qwen2.5-1.5B-Instruct", device_map="auto")

def run_agent(question, max_steps=4):
    messages = [
        {"role": "system", "content": TOOL_SPEC},
        {"role": "user", "content": question},
    ]
    for _ in range(max_steps):
        out = gen(messages, max_new_tokens=120, do_sample=False)
        reply = out[0]["generated_text"][-1]["content"].strip()

        try:
            call = json.loads(reply)           # did it ask for a tool?
        except json.JSONDecodeError:
            return reply                       # no → final answer

        tool = TOOLS.get(call["tool"])
        result = tool(**call["args"]) if tool else "Error: unknown tool"

        messages.append({"role": "assistant", "content": reply})
        messages.append({"role": "user",
                         "content": f"Tool result: {result}"})
    return "Stopped: too many steps."

```

5.4 Step 3: watch it work

Python

```
print(run_agent("What is 1234 * 17, and how many words are in this question?"))
```

The model first emits a `calculator` call, gets the product, then a `word_count` call, then writes the final sentence combining both — exactly the loop from the start of the chapter.

Tip

Smaller models follow strict output formats less reliably. Two cheap fixes: keep the tool list short, and give one worked example in the system prompt. If a model still drifts, step up one size (here 1.5B over 0.5B) before adding frameworks.

5.5 Using a framework

Hand-rolling teaches the mechanics; in production a small framework handles the parsing, retries, and tracing. The loop is identical — only the boilerplate disappears.

```
pip install smolagents
```

Python

```
from smolagents import CodeAgent, tool, TransformersModel

@tool
def calculator(expression: str) → str:
    """Evaluate arithmetic. Args: expression: the expression to evaluate."""
    return str(eval(expression))

agent = CodeAgent(
    tools=[calculator],
    model=TransformersModel(model_id="Qwen/Qwen2.5-1.5B-Instruct"),
)
print(agent.run("What is 1234 * 17?"))
```

5.6 Recap and exercise

You built an agent from first principles — the describe/call/execute/feed-back loop — and saw the same thing through a framework.

Exercise. Add a third tool, `get_time()`, that returns the current time, and ask the agent a question that needs two of the three tools. Confirm it chains the calls correctly.

Quantization for Small Hardware

In this chapter

Why quantization is the single biggest lever for running models on modest hardware; what 8-bit and 4-bit actually mean; loading a model in 4-bit with `bitsandbytes`; and exporting to GGUF for `llama.cpp` and Ollama.

6.1 The memory problem

Model weights are numbers. By default each is a 16-bit float, so a 7B-parameter model needs roughly $7\text{B} \times 2 \text{ bytes} \approx 14\text{GB}$ just to load — before any activations. That does not fit a typical laptop GPU. **Quantization** stores each weight in fewer bits, shrinking the model so it fits, with surprisingly little quality loss.

Bits to gigabytes

Rough memory for a 7B model: **fp16** $\approx 14\text{GB}$, **8-bit** $\approx 7\text{GB}$, **4-bit** $\approx 3.5\text{GB}$. Four-bit is the sweet spot for local use: it fits an 8 GB GPU and keeps most of the quality for chat, RAG, and agent work.

6.2 4-bit loading with bitsandbytes

This is the most common path: quantize *on load*, directly in `transformers`. No separate conversion step.

```
pip install bitsandbytes accelerate
```

Python

```
import torch
from transformers import (
    AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig,
)
```

```

quant_cfg = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",           # 4-bit NormalFloat, good default
    bnb_4bit_compute_dtype=torch.float16, # math still done in fp16
    bnb_4bit_use_double_quant=True,      # quantize the quantization constants
)

model_id = "Qwen/Qwen2.5-3B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id, quantization_config=quant_cfg, device_map="auto",
)

print(model.get_memory_footprint() / 1e9, "GB") # far below fp16

```

The model is used exactly like any other — generation code does not change.

Python

```

msgs = [{"role": "user", "content": "Name two quantization formats."}]
ids = tokenizer.apply_chat_template(
    msgs, add_generation_prompt=True, return_tensors="pt").to(model.device)
out = model.generate(ids, max_new_tokens=60)
print(tokenizer.decode(out[0][ids.shape[-1]:], skip_special_tokens=True))

```

Tip

nf4 (NormalFloat-4) usually beats plain int4 for quality at the same size, and **double quantization** saves a little more memory for free. These two defaults are good for almost everything; change them only after measuring.

6.3 GGUF for llama.cpp and Ollama

bitsandbytes quantizes for the Python/GPU path. For the llama.cpp ecosystem (CPU-friendly, powers Ollama and LM Studio) the format is **GGUF**, with quantization levels like Q4_K_M.

```

## Many models are already published as GGUF on the Hub.
# To convert one yourself with llama.cpp tooling:
python convert_hf_to_gguf.py ./my-model --outfile model.f16.gguf
./llama-quantize model.f16.gguf model.Q4_K_M.gguf Q4_K_M

```

You can then run that GGUF file directly, or import it into Ollama with a two-line Modelfile:

```

## Modelfile
FROM ./model.Q4_K_M.gguf

# Build and run it
ollama create my-model -f Modelfile
ollama run my-model "Hello from a quantized local model."

```


Which path do I use?

Training or staying in Python on a GPU → **bitsandbytes 4-bit**. Shipping a single portable file that runs anywhere, including CPU-only laptops → **GGUF** via `llama.cpp`/Ollama.

6.4 Recap and exercise

You learned what quantization trades and why, loaded a 3B model in 4-bit on modest hardware, and saw how GGUF serves the CPU-friendly local ecosystem.

Exercise. Load the same model in fp16 and in 4-bit, print `get_memory_footprint()` for each, and compare a few generations. Decide whether the quality difference is noticeable for your task.

Serving and Deployment

In this chapter

Turning a notebook model into a service others can call; a minimal but production-shaped FastAPI endpoint with streaming; serving through Ollama's built-in API; and the practical knobs — concurrency, batching, and health checks.

7.1 From script to service

Everything so far ran in a notebook. To put a model behind a product you need an **HTTP endpoint**: a long-running process that loads the model once and answers many requests. We will build one with FastAPI, then show the zero-code alternative via Ollama.

7.2 A FastAPI generation service

```
pip install fastapi uvicorn
```

Load the model *once* at startup, not per request — loading is the slow part.

Python

```
# file: server.py
from fastapi import FastAPI
from pydantic import BaseModel
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

app = FastAPI(title="Local SLM API")

MODEL_ID = "Qwen/Qwen2.5-0.5B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, torch_dtype=torch.float16, device_map="auto"
)
```

```

class Query(BaseModel):
    prompt: str
    max_new_tokens: int = 200

@app.get("/health")          # cheap check for load balancers
def health():
    return {"status": "ok", "model": MODEL_ID}

@app.post("/generate")
def generate(q: Query):
    msgs = [{"role": "user", "content": q.prompt}]
    ids = tokenizer.apply_chat_template(
        msgs, add_generation_prompt=True, return_tensors="pt"
    ).to(model.device)
    out = model.generate(ids, max_new_tokens=q.max_new_tokens)
    text = tokenizer.decode(out[0][ids.shape[-1]:], skip_special_tokens=True)
    return {"response": text}

```

Run it and call it:

```

uvicorn server:app --host 0.0.0.0 --port 8000

# In another terminal:
curl -X POST http://localhost:8000/generate \
  -H "Content-Type: application/json" \
  -d '{"prompt": "Define quantization in one line."}'

```

Load once

The model is loaded at import time, so every request reuses it. Never load inside the request handler — that would re-read gigabytes of weights on every call and make the service unusably slow.

7.3 Streaming responses

For chat UIs, stream tokens instead of waiting for the full answer. FastAPI's `StreamingResponse` plus a threaded generation streamer does it.

Python

```

from fastapi.responses import StreamingResponse
from transformers import TextIteratorStreamer
from threading import Thread

@app.post("/stream")
def stream(q: Query):
    msgs = [{"role": "user", "content": q.prompt}]
    ids = tokenizer.apply_chat_template(
        msgs, add_generation_prompt=True, return_tensors="pt"
    ).to(model.device)

    streamer = TextIteratorStreamer(

```

```

tokenizer, skip_prompt=True, skip_special_tokens=True)
kwargs = dict(inputs=ids, max_new_tokens=q.max_new_tokens, streamer=streamer)
Thread(target=model.generate, kwargs=kwargs).start()

def token_iter():
    for token in streamer:
        yield token
return StreamingResponse(token_iter(), media_type="text/plain")

```

7.4 The no-code path: Ollama serve

If you do not need custom endpoints, Ollama already *is* a server with an OpenAI-compatible API. Start it once and any OpenAI client library can talk to it by pointing at the local URL.

```
ollama serve # exposes http://localhost:11434/
```

Python

```

from openai import OpenAI

client = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
resp = client.chat.completions.create(
    model="qwen2.5:0.5b",
    messages=[{"role": "user", "content": "Hello from the OpenAI client!"}],
)
print(resp.choices[0].message.content)

```

Tip

For higher throughput with many concurrent users on a GPU, look at **vLLM**, which adds continuous batching and a drop-in OpenAI-compatible server (`vllm serve <model>`). FastAPI is perfect for single-model, moderate-traffic local services; vLLM is for scale.

7.5 Recap and exercise

You wrapped a model in a FastAPI service with health, generation, and streaming endpoints, and saw the zero-code Ollama alternative.

Exercise. Add a `/summarize` endpoint that accepts long text and returns a three-sentence summary by composing a fixed instruction with the input. Test it with `curl`.

Capstone: A Local Document Assistant

In this chapter

Combining everything — a quantized model, RAG over your documents, a tool-using agent, and a served endpoint — into one local assistant that answers questions about your files and can do arithmetic on the way. This is the whole book in one program.

8.1 What we are building

A single service that:

1. loads a **quantized** model (Chapter 6) so it fits modest hardware;
2. retrieves context from your documents via **RAG** (Chapter 4);
3. can call a **tool** when the question needs computation (Chapter 5);
4. is exposed over **HTTP** (Chapter 7).

We assemble it from the pieces you already wrote, so each block should look familiar.

8.2 Part 1: the quantized model

Python

```
import torch
from transformers import (
    AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig, pipeline,
)

quant = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
)
```

```

MODEL_ID = "Qwen/Qwen2.5-1.5B-Instruct"
tok = AutoTokenizer.from_pretrained(MODEL_ID)
model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID, quantization_config=quant, device_map="auto")
gen = pipeline("text-generation", model=model, tokenizer=tok)

```

8.3 Part 2: the retrieval index

Python

```

import lancedb
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

def build_index(texts):
    vecs = embedder.encode(texts, normalize_embeddings=True)
    db = lancedb.connect("./capstone_db")
    rows = [{"text": t, "vector": v.tolist()} for t, v in zip(texts, vecs)]
    return db.create_table("kb", data=rows, mode="overwrite")

# Replace with the chunks of your own documents.
table = build_index([
    "Quantization to 4-bit lets a 7B model run in about 3.5 GB.",
    "LoRA trains small adapters, updating under 1% of parameters.",
    "RAG injects retrieved document chunks into the prompt at query time.",
])

def retrieve(question, k=3):
    qv = embedder.encode(question, normalize_embeddings=True)
    return [h["text"] for h in table.search(qv.tolist()).limit(k).to_list()]

```

8.4 Part 3: the tool

Python

```

import json

def calculator(expression: str) → str:
    allowed = set("0123456789+-*/(). ")
    return str(eval(expression)) if set(expression) ≤ allowed else "invalid"

TOOLS = {"calculator": calculator}

```

8.5 Part 4: the orchestrator

This is the brain: retrieve context, let the model optionally call the tool, then produce a grounded final answer.

Python

```
SYSTEM = """You answer questions about the user's documents.
Use the provided context. If a calculation is needed, reply with ONLY:
{"tool": "calculator", "args": {"expression": "..."}}
Otherwise reply with a normal answer."""

def assistant(question):
    context = "\n".join(retrieve(question))
    messages = [
        {"role": "system", "content": SYSTEM},
        {"role": "user",
         "content": f"Context:\n{context}\n\nQuestion: {question}"},
    ]
    for _ in range(3):
        reply = gen(messages, max_new_tokens=160,
                    do_sample=False)[0]["generated_text"][-1]["content"].strip()
        try:
            call = json.loads(reply)
        except json.JSONDecodeError:
            return reply  # final grounded answer
        result = TOOLS[call["tool"]](**call["args"])
        messages.append({"role": "assistant", "content": reply})
        messages.append({"role": "user", "content": f"Tool result: {result}"})
    return reply
```

Python

```
print(assistant("How much memory does a 4-bit 7B model need?"))
print(assistant("If LoRA updates 0.8% of 1.5 billion params, how many is that? "
                "Use the calculator."))
```

8.6 Part 5: serve it

Wrap the assistant function in the FastAPI pattern from Chapter 7 and you have a deployable local document assistant.

Python

```
# file: app.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Local Document Assistant")

class Ask(BaseModel):
    question: str

@app.post("/ask")
def ask(a: Ask):
    return {"answer": assistant(a.question)}
```

```
uvicorn app:app --port 8000
curl -X POST localhost:8000/ask -H "Content-Type: application/json" \
  -d '{"question": "What does RAG do?"}'
```

You built the whole stack

Quantized model + retrieval + tools + serving — the four pillars of practical SLM work, in one file. Every production system is an elaboration of these same pieces: better chunking, more tools, sturdier serving, evaluation, and monitoring.

8.7 Where to go next

- **Evaluation.** Build a small test set and score answers, so changes are measured, not guessed.
- **Better retrieval.** Add re-ranking and metadata filters.
- **Fine-tune the format.** Use Chapter 3 to lock the assistant's tone and structure.
- **Scale serving.** Move from FastAPI to vLLM when traffic grows.

Final exercise. Replace the toy knowledge base with a folder of your own documents, add one more tool relevant to your domain, and deploy the assistant locally. You now have a private, controllable AI application that runs entirely on your own hardware — the goal we set in the preface.